

Capstone Presentation Script

Video Link (publicly accessible with msu.edu email):

https://michiganstate-my.sharepoint.com/personal/saxenaud_msu_edu/_layouts/15/stream.aspx?id=%2Fpersonal%2Fsaxenaud%5Fmsu%5Fedu%2FDocuments%2FCSE%20498%20Initial%20Classes%20Video%2Emp4&referrer=StreamWebApp%2EWeb&referrerScenario=AddressBarCopied%2Eview%2E0b8ff9a4%2Dd5d9%2D436f%2Db1eb%2Dc166f360ae9b

WebCanvas - Maksim

Hello, my name is Maksim Savich. The class that I focused on for the last few weeks was the WebCanvas class.

This class is a wrapper class for the canvas DOM element. It implements most of the general functionality of the canvas element using Emscripten's val header to interact with the JavaScript API. Fortunately for me, my job was made easy since the canvas element implementation already exists and is tested on the JavaScript side. Therefore, there is no need to recreate the canvas functionality on the C++ side. So, I was able to use the MDN documentation for writing the functionality.

The WebCanvas class enables simple calls with well documented functions on the C++ side that could be used anywhere in the project. The class also offers safety checks to ensure that inputs do not result in undefined behavior. These inputs can be checked at compile time. We are also able to improve efficiency or add functionality without large refactors later on when developing the project.

Change Slide

Instead of making JavaScript handle all of the work done in the WebCanvas class, the class does store some information like the background color, canvas location, pen color, height, width etc. This way, we can reduce the overhead of querying JavaScript for information about a specific element. Of course, this is just an implementation choice. If we rarely ever query the JavaScript for information about an element, then we can shed the memory use of storing the information about a WebCanvas object. The class also stores a reference to the DOM element itself. Storing a reference about the DOM element is useful for later calls to the WebCanvas element, especially during destruction. This way, the C++ side can handle the lifetime of each instantiated canvas element.

Change Slide

Let's jump into the implemented functions. To start simple, we have some simple getters for width, height, and location of the canvas. You might wonder, "location?" How do we track location when these elements are rendered in HTML? There is a translate method that allows a

developer to move the canvas around the page. It enables verification for unit testing and for whatever checks we may need to do in the future.

The developer can draw lines, rectangles, circles, polygons, text. They can load images into an image cache, draw from that cache, or draw when they call DrawImage. There are setters for pen color, background color, fill color, line width, font, and alpha. As stated previously, the developer can translate, rotate, scale, and resize the canvas. They can also utilize the save and restore functions to save a state of the canvas, change it, and then immediately bring it back.

Change Slide

Lastly, we need a way to synchronize elements with the next draw call/drawn frame. I created a wrapper for emscripten's request_animation_frame function to pass a callback function. This enables continuous animations or the ability to constantly show updates for things like movement. Here is this bouncing ball animation for example. This callback function isn't necessarily limited to the canvas element. We had some talks after implementation and will likely be moving it to a manager class that will be handling all of the WebElements.

Here are more examples of what can be done with the WebCanvas class. Of course this functionality is rather basic and the implementation was quick due to just having to call JavaScript. So we will be doing more work as features are requested.

Change Slide

WebLayout - Udbhav

Hi, my name is Udbhav. The class I implemented is WebLayout, which will be used for laying out other elements that are Web UI classes along a one-dimensional axis.

Slide change

WebLayout is a simple container encompassing the functionality of a [Flex](#) container, allowing children to be arranged along an axis with options for justifying and aligning them. Flex is a style of laying out elements in one direction that, as its name suggests, is flexible with elements being able to resize themselves and align in different ways.

Slide change

These are some example properties we can set in a flex container, allowing us to create a variety of layouts. A flex container can be both vertical or horizontal. Align sets alignment along the cross axis, while justify sets alignment along the main axis. These small snippets display how we would set those properties within WebLayout

Slide change

WebLayout currently tracks its children by using a vector of shared pointers, which may change as we settle on a lifecycle management methodology for UI elements.

Slide change

(Describe full API as of initial implementation)

Slide change

Since WebLayout itself is a descendant of WebElement, they can also be nested to create 2 dimensional layouts.

Slide change

WebTextbox - Varun

Hello, my name is Varun, and I was responsible for developing the WebTextbox class. While the WebCanvas class handles the graphical simulation part of the code, my code acts as the gateway between the user and the backend. Specifically, it acts as a kind of console of the ReplayDriver to output all of the action logs run by the simulation.

I built the WebTextbox using embind in order to directly wrap the 'div' elements in the HTML. This basically allows the code to not only create, but to change and style the DOM without writing a single line in Java.

Slide Change

There were 2 main challenges I had faced. The first was performance, and overall stability. Since the data logger could theoretically stream thousands of events, like spawning agents, pathfinding, state changes, etc, adding all of that text to the DOM would eventually bloat the memory usage to an unstable amount.

To fix this, I implemented a circular buffer system, wherein the class automatically detects when the log's size exceeds 50KB, and gets rid of the oldest entries, effectively ensuring that it can run without much degradation to its performance over time.

Slide Change

The second, and more important challenge, was security. Now, I'm not really one for security, in fact, I suck at even just thinking about implementing security features. But nonetheless, it had to be done, so I tried my best. So let's say we are going to be displaying raw data from agents,

having the names of agents able to be set to malicious, executable code is basically asking for an injection.

For this, I explicitly designed the class to use the “innerText” property instead of “innerHTML”, which (from what little I know), gets rid of any Cross-Site Scripting attacks, strengthening the code.

Slide Change

Finally, in order to align with code set by my team, I specifically engineered WebTextbox to be Headless-aware. The class basically detects if it's running in a browser or server environment, like [Node.js](#). If no browser is found, it switches to an internal buffer. This now allows us to run aggressive testing, as well as stress tests directly through the terminal, without breaking the build.

WebImage - Mariana

I'm Mariana and I worked on implementing the WebImage class that manages image elements on our web interface.

Similar to the other classes, my implementation is a wrapper of a DOM element, in this case an image. Here I have a simplified version of my header file so that you can easily see the different parts being used to implement this.

We have the private section storing our image ID, source path, dimensions, position, and the `emscripten::val` that is used to handle the actual DOM element. The constructor and destructor are used to handle the creation and destruction of the image tag on the DOM and since each instance owns one DOM node, the copy and assignment operators are deleted.

The setters are able to update the image via `emscripten::val`, while the getters will return the current C++ state of the image. There are also some functions included to check the image for loading and error status since images load asynchronously in the browser.

Slide Change

The constructor works by using an `emscripten::val` to access the dom, verifies the ID is unique, then directly creates the image tag. It sets the source and ID as given, adds it to the document body, then enables absolute positioning for later position setting.

Slide Change

The setters are implemented by first making any necessary assertions, storing the new set values in the C++ member variables, then syncing those variables to the DOM via the `emscripten::val`.

Slide Change

Here I have a usage example for a car agent, which you can see is straightforward C++ with no need to touch any JavaScript. This class is useful as it allows you to create the image object, set the properties, update as needed, and when you're done the DOM element cleans up itself.

WebButton - Fatima

Hi, my name is Fatima, and the class I worked on is WebButton.

WebButton is a C++ abstraction that represents a user interface button. It stores key properties such as the button's label, a unique identifier, whether the button is visible, whether it is enabled, and an optional callback that runs when the button is clicked.

Rather than directly interacting with the browser, the class manages its state entirely in C++. This makes it easy to reuse across the project and allows it to be tested outside of a web environment.

Slide Change:

The constructor initializes the button with a label and an ID. It also validates that the ID is not empty, ensuring that each button can be uniquely identified.

All of the button's data is stored internally in C++, including the label and ID. This design keeps the class independent from the underlying web implementation while still maintaining the information needed to link it to the interface.

Slide Changes:

WebButton provides functions to control whether the button is visible and whether it is enabled. These two properties are managed independently.

For example, a button can be visible but disabled, meaning the user can see it but cannot interact with it.

These operations simply update internal variables, making the class safe, predictable, and easy to test without requiring a browser.

Slide Change:

A function can be registered using `SetOnClick`, which defines what should happen when the button is pressed.

When `Click` is called, the callback only runs if the button is both visible and enabled. This prevents unintended actions when the button should not be interactive.

This approach provides a clean and safe model for handling UI events, making `WebButton` suitable for simulation controls such as `Start`, `Pause`, or `Reset` buttons.